Machine Learning Practical Exam Codes

1. Iris Dataset – Import, Preprocess, Normalize

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_iris
from sklearn.preprocessing import LabelEncoder, MinMaxScaler

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

df['species'] = iris.target

df['species'] = df['species'].map({
    0:'setosa',
    1:'versicolor',
    2:'virginica'
})

print(df.head())

print(df.isnull().sum())

le = LabelEncoder()
df['species'] = le.fit_transform(df['species'])

scaler = MinMaxScaler()
df[df.columns[:-1]] = scaler.fit_transform(df[df.columns[:-1]])

print(df.head())
```

---

2. Academic Performance Dataset

```
import pandas as pd
import numpy as np

data = {
    'Name':['A','B','C','D'],
    'Math':[80, np.nan, 75, 90],
    'Science':[70, 85, np.nan, 88],
    'Attendance':[90, 85, 80, np.nan]
}

df = pd.DataFrame(data)

print(df)

print(df.isnull().sum())

df['Math'].fillna(df['Math'].mean(), inplace=True)
df['Science'].fillna(df['Science'].mean(), inplace=True)
df['Attendance'].fillna(df['Attendance'].mean(), inplace=True)

df['Math_log'] = np.log(df['Math'])

print(df)
```

---

3. Summary Statistics of Iris Dataset

```
import pandas as pd
from sklearn.datasets import load_iris
iris = load_iris()
```

```
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

print(df.groupby('species').describe())

print("Mean")
print(df.mean())

print("Standard Deviation")
print(df.std())

print("Percentile")
print(df.quantile([0.25,0.5,0.75]))
```

---

4. Linear Regression – Housing Dataset

```
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

data = fetch_california_housing()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predictions:", y_pred[:5])

mse = mean_squared_error(y_test, y_pred)
print("MSE:", mse)
```

---

5. Logistic Regression – Social Network Ads

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

df = pd.read_csv("Social_Network_Ads.csv")

X = df[['Age','EstimatedSalary']]
y = df['Purchased']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0)

model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)

print(cm)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
```

---

6. Naive Bayes on Iris Dataset

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1)

model = GaussianNB()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(confusion_matrix(y_test, y_pred))
print("Accuracy:", accuracy_score(y_test, y_pred))
```

---

7. NLP Preprocessing – TF-IDF

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer

text = "Machine learning is very useful in data science."

tokens = word_tokenize(text)
print(tokens)

stop_words = set(stopwords.words('english'))
filtered = [w for w in tokens if w.lower() not in stop_words]
print(filtered)

ps = PorterStemmer()
stemmed = [ps.stem(w) for w in filtered]
print(stemmed)

lm = WordNetLemmatizer()
lemm = [lm.lemmatize(w) for w in filtered]
print(lemm)

docs = [text]
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(docs)

print(vectorizer.get_feature_names_out())
print(X.toarray())
```

---

8. Titanic Histogram

```
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset('titanic')
plt.hist(df['fare'].dropna(), bins=20)
```

```
plt.title("Fare Distribution")
plt.xlabel("Fare")
plt.ylabel("Count")

plt.show()
```

---

9. Titanic Boxplot

```
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset('titanic')

sns.boxplot(x='sex', y='age', hue='survived', data=df)

plt.title("Age Distribution")
plt.show()
```

---

10. Iris Dataset – Histogram & Boxplot

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris

iris = load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

df.hist(figsize=(10,8))
plt.show()

for column in df.columns:
    sns.boxplot(y=df[column])
    plt.title(column)
    plt.show()
```

---

11. Accuracy Formula

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
```

---

12. Precision & Recall Formula

```
Precision = TP / (TP + FP)

Recall = TP / (TP + FN)
```
